

SISTEMI DISTRIBUITI E CLOUD COMPUTING

Anno Accademico 2025/2026

FinSight

Relazione Tecnica

Progettazione, gestione e analisi di portafogli finanziari



Vito Simone Goffredo

Matricola 277318

Prof. Domenico Talia

Prof. Loris Belcastro

Indice

1 Introduzione e scopo del progetto	3
2 Requisiti	3
2.1 Requisiti funzionali	3
2.2 Requisiti non funzionali	3
3 Architettura del sistema	4
3.1 Paradigma architetturale: il Monolite Modulare Containerizzato	4
3.2 Topologia di rete e difesa in profondità	4
3.3 Pattern Asynchronous Request-Reply (202 Accepted)	4
4 Stack tecnologico e motivazioni	5
5 Struttura del codice e organizzazione dei package	7
6 Gestione dei dati	8
6.1 Schema relazionale e modello di dominio	8
6.2 Dati semi-strutturati e audit trail (MarketDataSnapshot)	10
6.3 Persistenza dei report di AI generativa (AIReport)	11
6.4 Accesso ai dati: Repository Pattern ed evoluzione dello schema	11
7 Backend e servizio API	12
7.1 Routing RESTful e gateway applicativo	12
7.2 Data Transfer Object (DTO) e gestione degli errori	13
8 Elaborazione asincrona	14
8.1 Worker in background e condivisione del codebase	14
8.2 Ponte sync/async (Celery e asyncpkg)	15
8.3 Pattern di polling	16
9 Integrazione dei dati di mercato	16
9.1 Pattern Cache-Aside e graceful degradation	16
9.2 Analisi informative per singolo titolo	17
10 Motore quantitativo	18
11 Intelligenza artificiale come funzionalità	19
11.1 Il modulo GenerativeAIAnalyzer	19
11.2 Il pivot tecnologico sul provider e la mitigazione delle allucinazioni	19
11.3 Intelligenza effimera e pattern “Genera-poi-Verifica”	20
12 Sicurezza e controllo degli accessi	22
12.1 Autenticazione basata su JWT	22
12.2 Autorizzazione e Data Ownership	22
12.3 Protezione del gateway (Rate Limiting)	23
13 Frontend e dashboard	24

14 Observability, qualità e collaudo prestazionale	24
14.1 Structured Logging (JSON)	24
14.2 Esportazione delle metriche (Prometheus e Grafana)	25
14.3 Testing automatizzato e Continuous Integration	25
14.4 Chaos Engineering e diagnosi del collo di bottiglia	26
14.5 Ottimizzazione asincrona (Thread Offloading) e validazione comparativa	26
15 Metodologia di sviluppo assistita da AI	28
16 Limiti attuali	28
17 Sviluppi futuri	29
18 Conclusioni	29
Appendice A — Dichiarazione di utilizzo dell'AI generativa	29

1 Introduzione e scopo del progetto

FinSight è un'applicazione full-stack, containerizzata ed eseguita su cloud, per il **monitoraggio, la gestione e l'analisi di portafogli di titoli azionari**. L'utente può catalogare uno o più portafogli e le relative posizioni (titoli posseduti), aggiornarne le quotazioni interrogando provider di mercato esterni, calcolare automaticamente profitti/perdite e indicatori di rischio, e ottenere analisi testuali generate da un modello linguistico di grandi dimensioni (LLM).

Il progetto nasce in ambito accademico (esame di Sistemi Distribuiti e Cloud Computing), ma è stato ingegnerizzato con criteri da **prodotto pronto per la produzione**: separazione netta dei layer, elaborazione asincrona dei carichi pesanti, autenticazione e autorizzazione, resilienza verso i provider esterni, osservabilità e una pipeline di integrazione continua. L'obiettivo della presente relazione è descrivere il sistema realmente implementato — architettura, scelte progettuali e funzionalità — distinguendo, dove rilevante, ciò che è verificabile dal codice sorgente da ciò che è documentato o costituisce un risultato sperimentale misurato.

2 Requisiti

2.1 Requisiti funzionali

Il sistema soddisfa i seguenti requisiti funzionali:

- **Gestione utenti:** registrazione e autenticazione; ogni utente accede esclusivamente ai propri dati.
- **Gestione portafogli e posizioni:** creazione, lettura ed eliminazione di portafogli; inserimento, lettura ed eliminazione di posizioni (ticker, quantità, prezzo medio di carico, data, settore, mercato), anche tramite inserimento multiplo.
- **Integrazione dati di mercato:** recupero di quotazioni e serie storiche da un provider esterno (Yahoo Finance), in modalità asincrona.
- **Analisi quantitativa:** calcolo automatico di valore di mercato, profitti/perdite (P/L) assoluti e percentuali, pesi degli asset, rendimenti multi-finestra e volatilità di portafoglio.
- **Analisi generativa (AI):** produzione di report descrittivi sul portafoglio e di analisi per singolo titolo (spiegazione dell'andamento, sintesi di notizie, individuazione di titoli comparabili).
- **Visualizzazione interattiva:** dashboard con tabelle analitiche e grafici (incluso lo storico prezzi a finestre temporali selezionabili).

2.2 Requisiti non funzionali

- **Reattività:** le operazioni a latenza elevata (rete e inferenza LLM) non devono bloccare l'interfaccia.
- **Sicurezza:** accesso autenticato, autorizzazione per proprietà delle risorse, protezione del gateway dagli abusi.
- **Resilienza:** tolleranza ai malfunzionamenti dei provider esterni e degradazione controllata in caso di indisponibilità della cache.
- **Coerenza e precisione:** rappresentazione esatta dei valori finanziari (nessun errore di arrotondamento da virgola mobile).

- **Osservabilità e qualità:** log strutturati, metriche esportabili, test automatizzati e integrazione continua.
- **Portabilità:** esecuzione riproducibile tramite container.

3 Architettura del sistema

3.1 Paradigma architetturale: il Monolite Modulare Containerizzato

Per *FinSight* è stata scartata l'ipotesi di frammentare il dominio in una costellazione di servizi distribuiti a granularità fine, al fine di evitare i ben noti anti-pattern del *Distributed Monolith* (latenze di rete interne, tracing distribuito, complessità operativa) ingiustificati per il dominio in esame. Si è optato per un **Monolite Modulare Containerizzato**, supportato da un **worker asincrono**. Questa scelta combina due proprietà che, lette insieme, ne definiscono l'identità architetturale:

- **Codebase unica e condivisa.** Il servizio API e il worker condividono la stessa identica base di codice — modelli ORM, configurazione, layer di accesso ai dati, servizi. I due container (**api** e **worker**) buildano dal medesimo contesto (`./backend`) e si differenziano unicamente nel comando di avvio. Questo elimina la duplicazione di modelli e logica tipica della frammentazione spinta.
- **Esecuzione distribuita.** Pur essendo la codebase una sola, l'esecuzione è ripartita su processi distinti: il container **API** agisce da **gateway sincrono** (riceve il traffico HTTP, autentica e autorizza, valida i payload, instrada); il container **Worker** agisce da **motore asincrono in background** (esegue le chiamate lente verso i provider di mercato e i modelli LLM, e i calcoli onerosi).

Il risultato è la mantenibilità di un monolite con la possibilità di **scalare orizzontalmente il solo worker** in funzione del carico di elaborazione, senza moltiplicare la complessità di un'architettura a servizi indipendenti.

3.2 Topologia di rete e difesa in profondità

Il vincolo di virtualizzazione esclusiva tramite Docker Engine è stato sfruttato per progettare una rete segregata. Nel file di orchestrazione sono definite due reti virtuali distinte:

- **public_net** — rete esposta, a cui sono collegati i servizi raggiungibili dall'esterno: il servizio API e il client frontend, deputati alla ricezione del traffico HTTP.
- **private_net** — rete di backend isolata dall'host, a cui sono collegati i servizi applicativi e i datastore (API, Worker, PostgreSQL 16 e Redis 7). I datastore **non espongono alcuna porta sulla macchina host**.

Questa segregazione riduce la superficie di attacco e realizza un approccio **defence-in-depth** coerente con i principi Zero-Trust (isolamento di rete ed esposizione minima). Il Worker conserva capacità di *egress* per interrogare i provider esterni (**yfinance** per i dati di mercato e la piattaforma di inferenza **Groq** per i modelli linguistici), interfacciandosi con il core in modo asincrono tramite Redis. Complessivamente l'orchestrazione definisce **sette servizi** — **db** (PostgreSQL 16), **cache** (Redis 7), **api**, **worker**, **frontend**, **prometheus**, **grafana** — distribuiti sulle due reti.

3.3 Pattern Asynchronous Request-Reply (202 Accepted)

Per preservare la reattività dell'interfaccia ed evitare il blocco dei thread HTTP del gateway, le operazioni a lunga durata adottano il pattern **Asynchronous Request-Reply**:

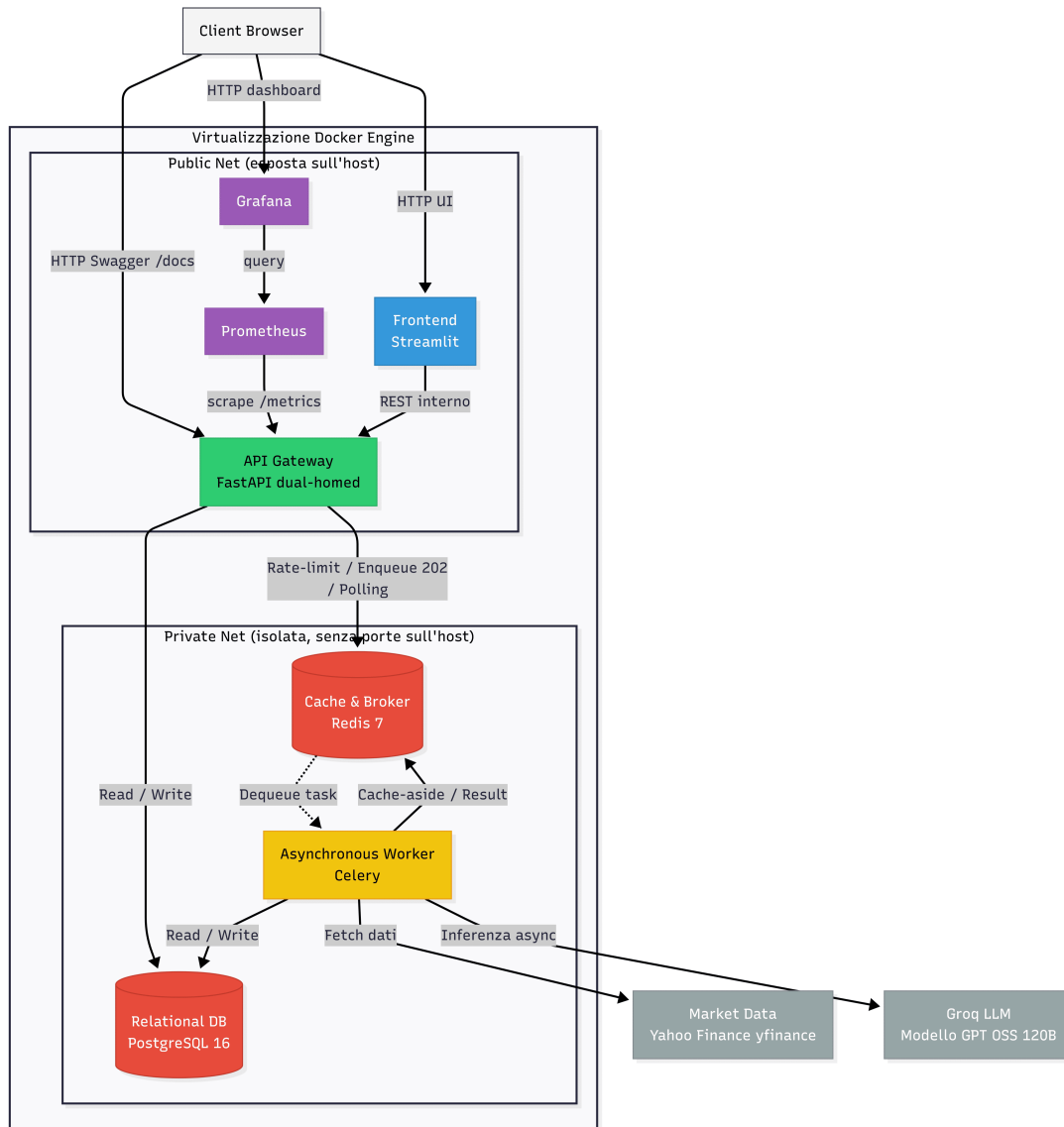


Figura 1: Topologia globale di FinSight: flusso dei dati e isolamento di rete tra i moduli applicativi, i datastore e lo stack di telemetria.

1. il client invoca l'endpoint dell'operazione (es. fetch dei dati o generazione di un report);
2. il gateway valida il payload e delega l'operazione a Redis sotto forma di messaggio;
3. il gateway risponde immediatamente con **HTTP 202 Accepted**, restituendo l'identificativo del task (**task_id**);
4. il Worker consuma il messaggio ed esegue in modo asincrono le chiamate di rete o l'inferenza;
5. il client si sincronizza tramite **polling** sull'endpoint di stato, finché il task transita in **SUCCESS** o **FAILURE**.

4 Stack tecnologico e motivazioni

Tutte le tecnologie elencate sono state verificate dai file **requirements.txt** e dal codice sorgente.

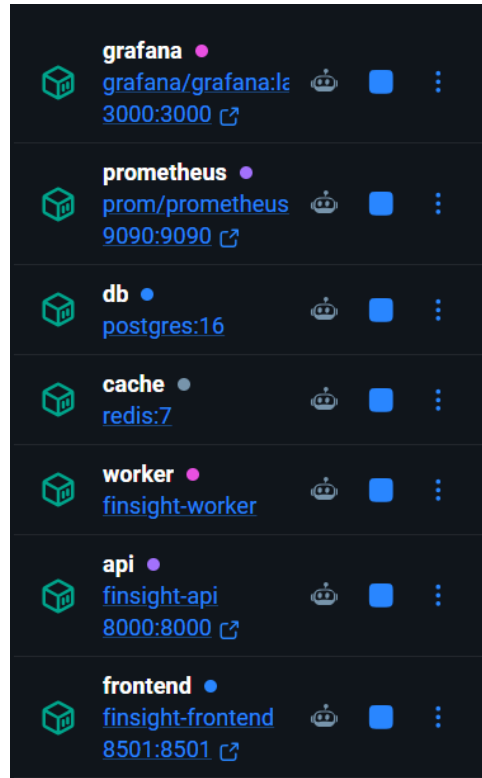


Figura 2: Stato di esecuzione dei 7 container Docker attivi e orchestrati via Docker Compose

Ambito	Tecnologia	Motivazione
Linguaggio	Python 3.11	Ecosistema maturo per web async, data analysis e AI
API backend	FastAPI 0.108 su Uvicorn (ASGI)	Async nativo, validazione integrata, OpenAPI automatico
Persistenza	SQLAlchemy 2.0 async + asyncpg, PostgreSQL 16	ORM tipizzato e I/O non bloccante; ACID + supporto JSONB
Migrazioni	Alembic (env asincrono)	Evoluzione deterministica e versionata dello schema
Validazione / DTO	Pydantic v2 + pydantic-settings	Validazione ad alte prestazioni (core in Rust)
Coda asincrona	Celery + Redis (broker e result backend)	Disaccoppiamento dei carichi pesanti
Cache	Redis 7 (cache-aside, TTL 900 s)	Riduzione latenza ed egress verso i provider
Dati di mercato	yfinance + pandas + numpy	Recupero quotazioni/storici ed elaborazione numerica
AI generativa	SDK openai (AsyncOpenAI) su Groq, modello GPT OSS 120B	Inferenza a bassa latenza e basso costo (vedi §11)
Auth/sicurezza	PyJWT (HS256), passlib[bcrypt], bcrypt 4.0.1, python-multipart	OAuth2 password flow, hashing robusto
Resilienza	fastapi-limiter	Rate limiting su Redis

Ambito	Tecnologia	Motivazione
Observability	<code>prometheus-fastapi-instrumentator</code> , Prometheus, Grafana, <code>python-json-logger</code>	Metriche pull-based e log JSON strutturati
Qualità	<code>pytest</code> , <code>pytest-asyncio</code> , <code>httpx</code> (ASGITransport), Locust	Test di integrazione e load testing
Frontend	Streamlit + Plotly + <code>pandas</code> + <code>requests</code>	Dashboard analitica rapida, client REST puro
Infrastruttura	Docker, Docker Compose, GitHub Actions	Riproducibilità e CI

5 Struttura del codice e organizzazione dei package

Il backend adotta un'**architettura a strati** che separa nettamente le responsabilità. L'organizzazione dei package (sotto `backend/app/`) è la seguente:

- `api/` — routing e dipendenze: i router REST versionati (`api/v1/endpoints/`) e il modulo `deps.py` con le dipendenze trasversali (autenticazione, ownership, identificatore di rate limiting).
- `core/` — fondamenta trasversali: configurazione (`config.py`), sicurezza (`security.py`), cache (`cache.py`), istanza Celery (`celery_app.py`), logging (`logging_config.py`), provider di mercato (`market_provider.py`), motore quantitativo (`quant.py`).
- `crud/` — il **Repository Pattern**: classe generica `CRUDBase` e repository specifici per entità.
- `db/` — accesso al database: engine asincrono e factory di sessioni (`session.py`), costruzione dell'URL (`url.py`), base dichiarativa (`base.py`).
- `models/` — i modelli ORM (entità relazionali).
- `schemas/` — i Data Transfer Object (DTO) Pydantic.
- `services/` — la logica applicativa di dominio, in particolare il modulo di AI generativa (`ai_service.py`).
- `tasks/` — i job Celery (`market.py`, `ai_tasks.py`).

Il frontend è un componente autonomo (`frontend/app.py`, dashboard Streamlit) containerizzato separatamente. Completano il progetto la configurazione di Prometheus (`infra/prometheus/`), lo script di load testing (`load_tests/locustfile.py`) e la pipeline CI (`.github/workflows/ci.yml`).

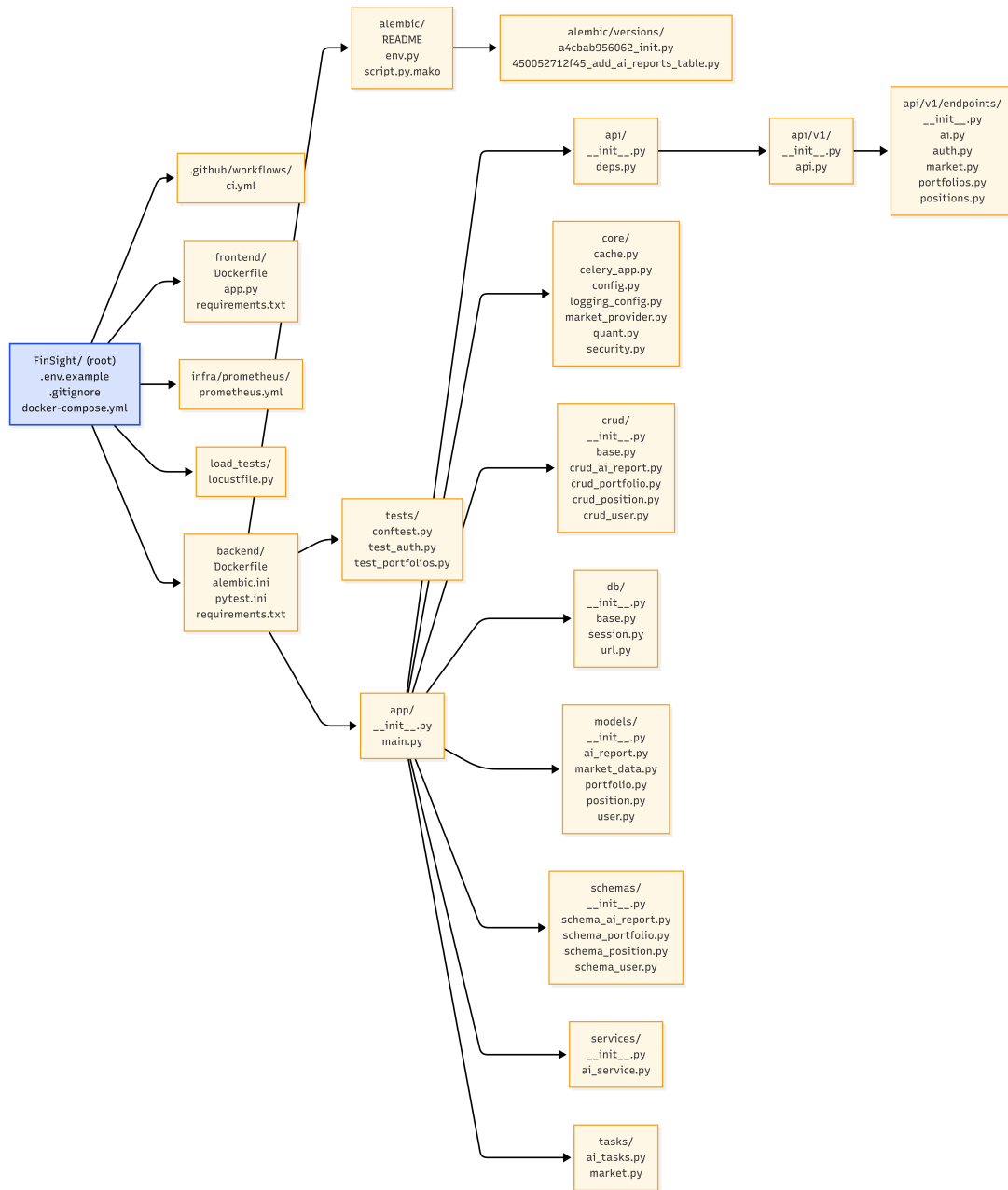


Figura 3: Struttura delle directory di FinSight.

6 Gestione dei dati

La strategia di persistenza è stata progettata per garantire le proprietà ACID — fondamentali in un contesto finanziario — mantenendo la flessibilità necessaria all'integrazione con sorgenti esterne. Il sistema si appoggia a **PostgreSQL 16**, interfacciato tramite **SQLAlchemy 2.0** in **modalità pienamente asincrona** (driver **asyncpg**).

6.1 Schema relazionale e modello di dominio

Lo schema entità-relazione è composto da **cinque modelli ORM**:

- **User** — l'utente (id, email, password con hash, data di creazione).
- **Portfolio** — il portafoglio, in relazione con l'utente proprietario.

- **Position** — la posizione (asset posseduto): ticker, quantità, prezzo medio di acquisto, data, settore, mercato.
- **MarketDataSnapshot** — lo storico grezzo dei payload restituiti dai provider (vedi §6.2).
- **AIReport** — i report di AI generativa persistiti (vedi §6.3).

Per garantire la precisione aritmetica ed evitare gli errori di arrotondamento tipici della virgola mobile, tutti i campi di natura finanziaria (quantità e prezzi) sono modellati con il tipo `Numeric(10, 4)`. Le chiavi primarie utilizzano `UUID`.

```

1  class Position(Base):
2      __tablename__ = "positions"
3
4      id: Mapped[uuid.UUID] = mapped_column(
5          UUID(as_uuid=True), primary_key=True, default=uuid.uuid4
6      )
7      portfolio_id: Mapped[uuid.UUID] = mapped_column(
8          UUID(as_uuid=True),
9          ForeignKey("portfolios.id", ondelete="CASCADE"),
10         index=True,
11         nullable=False,
12     )
13     ticker: Mapped[str] = mapped_column(String(16), index=True, nullable=False)
14     quantity: Mapped[Decimal] = mapped_column(Numeric(10, 4), nullable=False)
15     average_price: Mapped[Decimal] = mapped_column(Numeric(10, 4), nullable=False)
16     purchase_date: Mapped[datetime] = mapped_column(
17         DateTime(timezone=True), nullable=False
18     )
19     sector: Mapped[str | None] = mapped_column(String(128), nullable=True)
20     market: Mapped[str | None] = mapped_column(String(64), nullable=True)
21
22     portfolio: Mapped["Portfolio"] = relationship(back_populates="positions")
23
24     -----
25
26     class MarketDataSnapshot(Base):
27         __tablename__ = "market_data_snapshots"
28
29         id: Mapped[uuid.UUID] = mapped_column(
30             UUID(as_uuid=True), primary_key=True, default=uuid.uuid4
31         )
32         ticker: Mapped[str] = mapped_column(String(16), index=True, nullable=False)
33         raw_payload: Mapped[dict[str, Any]] = mapped_column(JSONB, nullable=False)
34         retrieved_at: Mapped[datetime] = mapped_column(
35             DateTime(timezone=True), server_default=func.now(), index=True, nullable=False
36         )

```

Listing 1: (position.py / market_data.py) Modelli ORM Position e MarketDataSnapshot. Definizione con `Numeric(10,4)`, `UUID` e colonna `JSONB`

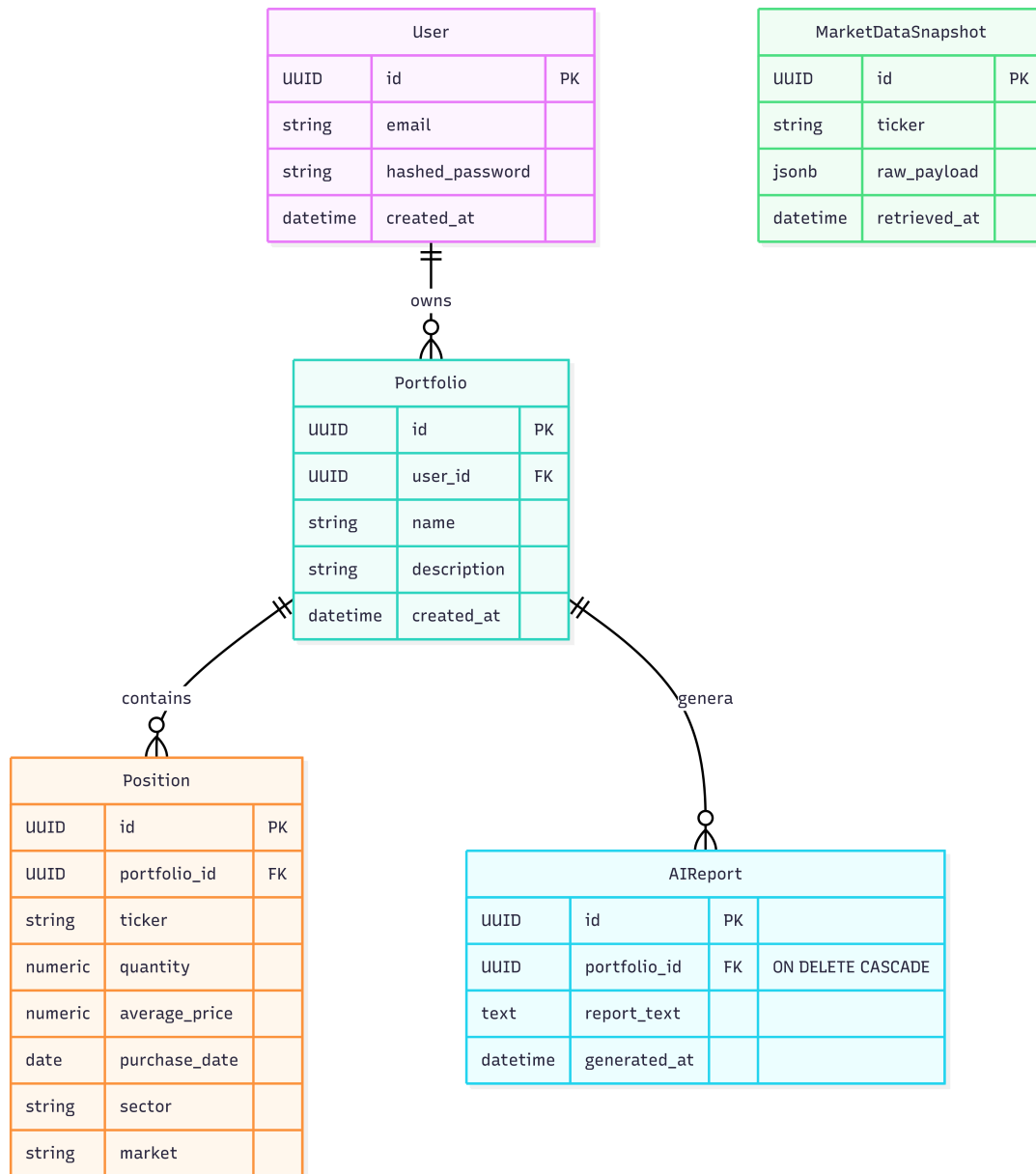


Figura 4: Schema Entità-Relazione (ER) del database PostgreSQL, evidenziando le dipendenze gerarchiche e l'uso di tipi ibridi (JSONB).

6.2 Dati semi-strutturati e audit trail (MarketDataSnapshot)

I provider finanziari restituiscono payload eterogenei e soggetti a frequenti mutazioni di schema. Per evitare costose migrazioni strutturali a ogni variazione del provider, è stata introdotta l'entità **MarketDataSnapshot**, dotata di una colonna **JSONB**. Questa entità funge da **audit trail** / **data lake grezzo**: a ogni recupero dati da Yahoo Finance, il Worker (`tasks/market.py`) vi scrive il payload integrale in modalità **append-only**, costruendo uno **storico immutabile e verificabile** di ciò che il provider ha restituito in un dato istante — proprietà preziosa in ottica di auditabilità e compliance. La colonna **JSONB** unisce la robustezza di un RDBMS alla flessibilità documentale tipica dei datastore NoSQL, conservando capacità di indicizzazione e interrogazione.

È opportuno precisare, in un'ottica di trasparenza progettuale, la **divisione dei ruoli tra i datastore**: le letture in tempo reale della pipeline analitica (Quant Engine e calcolo P/L) sono servite dalla **cache Redis** per privilegiare la velocità, mentre il database relazionale fun-

ge da **archivio permanente** dei payload provider. Nell'implementazione attuale, dunque, la pipeline analitica non interroga direttamente lo store JSONB: quest'ultimo privilegia flessibilità e auditabilità. Un'evoluzione orientata ad analytics storiche intensive introdurrebbe una tabella normalizzata delle osservazioni giornaliere (es. `HistoricalPrice`, con *open/high/low/close*, *adjusted close*, volume e provider, e vincolo univoco su ticker, data e provider).

6.3 Persistenza dei report di AI generativa (AIReport)

L'integrazione iniziale con i modelli LLM presentava un deficit di persistenza: i report risiedevano esclusivamente nel *Result Backend* temporaneo (Redis), soggetti alla volatilità del TTL. Per sanare questo debito tecnico è stata introdotta l'entità relazionale **AIReport**, legata da vincolo di chiave esterna al portafoglio di riferimento. Il salvataggio avviene in struttura **append-only**, garantendo un audit trail completo e consultabile storicamente dall'utente. Con l'aggiunta di **AIReport** il modello dati raggiunge le cinque entità sopra elencate.

6.4 Accesso ai dati: Repository Pattern ed evoluzione dello schema

Per disaccoppiare la logica di business dalle specifiche dell'ORM, è stato adottato il **Repository Pattern** (Data Access Object). Una classe generica astratta, **CRUDBase**, implementa le operazioni fondamentali (Create, Read, Update, Delete) operando su **TypeVar**; le classi concrete (es. `CRUDPortfolio`, `CRUDPosition`, `CRUDUser`, `CRUDAIReport`) ereditano dalla base e la estendono con metodi di dominio (es. `get_by_email`, `create_with_owner`, `get_by_portfolio`). I vantaggi sono la **testabilità** (mocking agevole) e la **sicurezza dei tipi** (in combinazione con i DTO Pydantic). La protezione contro la SQL injection non deriva dalla validazione Pydantic, bensì dall'uso sistematico dell'ORM e di statement parametrizzati, che evitano la costruzione di query per concatenazione di stringhe.

L'evoluzione dello schema è gestita con **Alembic**, configurato per operare in contesto totalmente asincrono. Ogni variazione genera un artefatto versionato, garantendo l'**idempotenza** dei deployment e la possibilità di **rollback** deterministici.

```

1  ModelType = TypeVar("ModelType", bound=Base)
2  CreateSchemaType = TypeVar("CreateSchemaType", bound=BaseModel)
3  UpdateSchemaType = TypeVar("UpdateSchemaType", bound=BaseModel)
4
5
6  class CRUDBase(Generic[ModelType, CreateSchemaType, UpdateSchemaType]):
7      """Repository generico asincrono, tipizzato su modello + schemi Pydantic."""
8
9      def __init__(self, model: type[ModelType]) -> None:
10         self.model = model
11
12         async def get(self, db: AsyncSession, id: uuid.UUID) -> ModelType | None:
13             return await db.get(self.model, id)
14
15         async def get_multi(
16             self, db: AsyncSession, *, skip: int = 0, limit: int = 100
17         ) -> Sequence[ModelType]:
18             result = await db.execute(select(self.model).offset(skip).limit(limit))
19             return result.scalars().all()
20
21         async def create(self, db: AsyncSession, *, obj_in: CreateSchemaType) ->
22             ModelType:
23             db_obj = self.model(**obj_in.model_dump())
24             db.add(db_obj)
25             await db.commit()
26             await db.refresh(db_obj)
27             return db_obj

```

```

28     async def update(
29         self, db: AsyncSession, *, db_obj: ModelType, obj_in: UpdateSchemaType
30     ) -> ModelType:
31         #aggiorna solo i campi davvero passati (PATCH)
32         update_data = obj_in.model_dump(exclude_unset=True)
33         for field, value in update_data.items():
34             setattr(db_obj, field, value)
35         db.add(db_obj)
36         await db.commit()
37         await db.refresh(db_obj)
38         return db_obj
39
40     async def remove(self, db: AsyncSession, *, id: uuid.UUID) -> ModelType |
None:
41         db_obj = await db.get(self.model, id)
42         if db_obj is not None:
43             await db.delete(db_obj)
44             await db.commit()
45         return db_obj

```

Listing 2: (crud-py) - La classe generica `CRUDBase` con i metodi asincroni tipizzati.

7 Backend e servizio API

7.1 Routing RESTful e gateway applicativo

Il punto di ingresso dell'applicazione è il **servizio API REST** (FastAPI), unico front-end applicativo, che si fa carico di access control (autenticazione e autorizzazione), rate limiting e validazione/orchestrizzazione del traffico verso il layer di Repository. Pur svolgendo funzioni tipiche di un API gateway, il servizio **non** implementa un reverse proxy dedicato né l'aggregazione di servizi a valle; nel seguito vi si fa riferimento, per brevità, come *gateway applicativo*. È stato adottato un **API Versioning su URI** (es. `/api/v1/portfolios`) per preservare la retrocompatibilità. FastAPI genera nativamente la documentazione **OpenAPI (Swagger UI)**, offrendo un *API Contract* sempre allineato al codice in esecuzione.

auth			^
POST	/api/v1/auth/register	Register	▼
POST	/api/v1/auth/login	Login	▼
portfolios			^
POST	/api/v1/portfolios/	Create Portfolio	🔒 ▼
GET	/api/v1/portfolios/	Read Portfolios	🔒 ▼
GET	/api/v1/portfolios/{portfolio_id}	Read Portfolio	🔒 ▼
DELETE	/api/v1/portfolios/{portfolio_id}	Delete Portfolio	🔒 ▼
positions			^
POST	/api/v1/positions/	Create Position	🔒 ▼
GET	/api/v1/positions/portfolio/{portfolio_id}	Read Positions By Portfolio	🔒 ▼
PATCH	/api/v1/positions/{position_id}	Update Position	🔒 ▼
DELETE	/api/v1/positions/{position_id}	Delete Position	🔒 ▼
market			^
POST	/api/v1/market/holdings/{ticker}	Request Eff Holdings	🔒 ▼
POST	/api/v1/market/fetch/{ticker}	Fetch Market Data	🔒 ▼
GET	/api/v1/market/status/{task_id}	Get Task Status	🔒 ▼
GET	/api/v1/market/prices/{portfolio_id}	Get Portfolio Prices	🔒 ▼
POST	/api/v1/market/dividends/{ticker}	Request Dividends	🔒 ▼
POST	/api/v1/market/analyst/{ticker}	Request Analyst Targets	🔒 ▼
ai			^
POST	/api/v1/ai/report/{portfolio_id}	Request Portfolio Report	🔒 ▼
POST	/api/v1/ai/explain/{ticker}	Request Ticker Explanation	🔒 ▼
GET	/api/v1/ai/reports/{portfolio_id}	Read Portfolio Reports	🔒 ▼
POST	/api/v1/ai/news/{ticker}	Request News Summary	🔒 ▼
POST	/api/v1/ai/peers/{ticker}	Request Sector Peers	🔒 ▼
default			^
GET	/metrics	Metrics	▼
GET	/health	Health	▼

Figura 5: Interfaccia Swagger UI autogenerata da FastAPI.

7.2 Data Transfer Object (DTO) e gestione degli errori

L'intero flusso di dati in ingresso e in uscita è vincolato a schemi rigidi implementati come **DTO Pydantic v2**. Qualsiasi disallineamento (tipi scorretti, campi obbligatori mancanti) viene intercettato a livello di gateway, generando un errore HTTP 422 (Unprocessable Entity) **prima** di raggiungere l'ORM. È inoltre presente un handler globale che converte la violazione di

vincoli del database (`IntegrityError`) in una risposta HTTP 400 esplicita. Un endpoint dedicato (`/health`) implementa un *readiness check* minimo, verificando la connettività con PostgreSQL tramite una query diagnostica (`SELECT 1`); il controllo è volutamente minimale (probe verso Redis e Worker sarebbero un'estensione naturale).

```

1 router = APIRouter()
2
3
4 @router.post("/", response_model=PortfolioRead, status_code=status.
    HTTP_201_CREATED)
5 async def create_portfolio(
6     obj_in: PortfolioCreate,
7     current_user: Annotated[User, Depends(get_current_user)],
8     db: Annotated[AsyncSession, Depends(get_session)],
9 ) -> Portfolio:
10     return await crud_portfolio.create_with_owner(
11         db, obj_in=obj_in, owner_id=current_user.id
12     )
13
14
15 @router.get("/", response_model=list[PortfolioRead])
16 async def read_portfolios(
17     current_user: Annotated[User, Depends(get_current_user)],
18     db: Annotated[AsyncSession, Depends(get_session)],
19     skip: int = 0,
20     limit: int = 100,
21 ) -> Sequence[Portfolio]:
22     return await crud_portfolio.get_by_owner(
23         db, owner_id=current_user.id, skip=skip, limit=limit
24     )
25
26
27 @router.get("/{portfolio_id}", response_model=PortfolioRead)
28 async def read_portfolio(
29     portfolio: Annotated[Portfolio, Depends(get_owned_portfolio)],
30 ) -> Portfolio:
31     return portfolio
32
33
34 @router.delete("/{portfolio_id}", status_code=status.HTTP_204_NO_CONTENT)
35 async def delete_portfolio(
36     portfolio: Annotated[Portfolio, Depends(get_owned_portfolio)],
37     session: Annotated[AsyncSession, Depends(get_session)],
38 ) -> None:
39     await session.execute(delete(AIReport).where(AIReport.portfolio_id ==
        portfolio.id))
40     await session.execute(delete(Position).where(Position.portfolio_id ==
        portfolio.id))
41     await session.delete(portfolio)
42     await session.commit()

```

Listing 3: Endpoint (`portfolios.py`): applicazione pratica di Dependency Injection, Data Transfer Object (DTO) e controllo di *ownership* sulle risorse.

8 Elaborazione asincrona

8.1 Worker in background e condivisione del codebase

L'integrazione di funzionalità computazionalmente onerose (serie storiche e inferenza LLM) impone un sistema di accodamento dei task: è stato integrato **Celery**, con **Redis** come Message Broker e Result Backend. Come descritto in §3.1, il servizio `api` e il worker **condividono il**

medesimo codebase (build da `./backend`), differenziandosi solo nel comando di avvio. Questa scelta elimina la duplicazione di modelli ORM, configurazioni e layer CRUD, e consente al Worker di scalare orizzontalmente in modo indipendente.

8.2 Ponte sync/async (Celery e asyncpg)

L'esecuzione di task asincroni legati al database all'interno del contesto sincrono di Celery richiede un'attenta gestione dell'event loop, poiché il driver **asyncpg** solleva criticità se le connessioni vengono condivise tra thread o loop differenti. Per prevenire la corruzione del pool e i deadlock, l'architettura istanzia un **engine task-local** per ogni job, utilizza un **NullPool** (che disabilita il connection pooling di SQLAlchemy) e avvia/termina esplicitamente il ciclo di vita della connessione all'interno di `asyncio.run()`. Questa combinazione garantisce un ciclo di vita isolato per ciascuna connessione, assicurando la thread-safety durante l'ingestione massiva di dati.

```
1 logger = get_task_logger(__name__)
2
3
4 async def _store_snapshot(ticker: str, payload: dict[str, Any]) -> str:
5     engine = create_async_engine(get_async_database_url(), poolclass=NullPool)
6     try:
7         async with AsyncSession(engine, expire_on_commit=False) as session:
8             snapshot = MarketDataSnapshot(ticker=ticker, raw_payload=payload)
9             session.add(snapshot)
10            await session.commit()
11            await session.refresh(snapshot)
12            return str(snapshot.id)
13    finally:
14        await engine.dispose()
15
16
17 @celery_app.task(name="fetch_market_data_task")
18 def fetch_market_data_task(ticker: str) -> dict[str, str]:
19     ticker_info = fetch_ticker_info(ticker)
20     if not ticker_info or ticker_info.get("regularMarketPrice") is None:
21         raise ValueError(f"Invalid or unknown ticker: {ticker}")
22     snapshot_id = asyncio.run(_store_snapshot(ticker, ticker_info))
23     return {"message": "Data fetched successfully", "snapshot_id": snapshot_id}
24
25 @celery_app.task(name="fetch_etf_holdings_task")
26 def fetch_etf_holdings_task(ticker: str) -> dict:
27     ticker = ticker.upper()
28     info = fetch_ticker_info(ticker) or {}
29     quote_type = (info.get("quoteType") or "").upper()
30     if quote_type not in ("ETF", "MUTUALFUND"):
31         return {"status": "success", "ticker": ticker, "holdings": [],
32                "message": f"{ticker} non e' un ETF/fondo: nessuna composizione disponibile."}
33     holdings = fetch_etf_holdings(ticker)
34     if not holdings:
35         return {"status": "success", "ticker": ticker, "holdings": [],
36                "message": f"Composizione non disponibile per {ticker}."}
37     return {"status": "success", "ticker": ticker,
38            "name": info.get("longName") or info.get("shortName") or ticker,
39            "holdings": holdings}
40
41 @celery_app.task(name="fetch_dividends_task")
42 def fetch_dividends_task(ticker: str) -> dict:
43     ticker = ticker.upper()
44     dividends = fetch_dividends(ticker)
45     if not dividends:
46         return {"status": "success", "ticker": ticker, "dividends": [],
```

```

47         "message": f"Nessuno storico dividendi per {ticker} (il titolo
potrebbe non distribuirne)."}
48     return {"status": "success", "ticker": ticker, "dividends": dividends}
49
50 @celery_app.task(name="fetch_analyst_task")
51 def fetch_analyst_task(ticker: str) -> dict:
52     ticker = ticker.upper()
53     analyst = fetch_analyst_targets(ticker)
54     if analyst.get("target_mean") is None:
55         return {"status": "success", "ticker": ticker, "analyst": None,
56                 "message": f"Dati analisti non disponibili per {ticker}."}
57     return {"status": "success", "ticker": ticker, "analyst": analyst}

```

Listing 4: Worker Celery (market.py): pattern di esecuzione asincrona tramite `asyncio.run()` e gestione sicura delle connessioni al database con `NullPool`.

8.3 Pattern di polling

Per permettere al client di sincronizzarsi con lo stato di avanzamento, ricevuto l'HTTP 202 Accepted con il `task_id`, il client interroga iterativamente l'endpoint di stato (GET `/api/v1/market/status/{task_id}`), che consulta il Result Backend di Celery su Redis e restituisce lo stato corrente (PENDING, SUCCESS, FAILURE). A completamento avvenuto vengono restituiti i riferimenti ai record persistenti.

9 Integrazione dei dati di mercato

Il Worker si interfaccia con il provider esterno **yfinance** (Yahoo Finance) per il recupero di quotazioni e dati storici. Le interazioni sono centralizzate nel modulo `core/market_provider.py`, che espone wrapper per le diverse risorse (informazioni sul ticker, notizie — con normalizzazione del vecchio e nuovo schema del provider —, holdings di ETF, dividendi, target degli analisti).

9.1 Pattern Cache-Aside e graceful degradation

L'operatività del Worker genera traffico in *egress* verso il provider. Per ottimizzare la banda e ridurre il rischio di IP ban è stato integrato un pattern **Cache-Aside** su Redis:

- in caso di **Cache Hit**, i payload (serializzati in JSON, inclusi i DataFrame `pandas`) vengono restituiti istantaneamente;
- in caso di **Cache Miss**, il Worker esegue la chiamata di rete, processa il dato e lo persiste in cache con TTL deterministico (es. 900 s per le quotazioni);
- le operazioni di caching sono avvolte in blocchi `try/except`: un'eventuale indisponibilità di Redis degrada il sistema in modo trasparente (**graceful degradation**), bypassando la cache senza interrompere il servizio.

Per tollerare i blackout transitori del provider gratuito, l'architettura implementa inoltre una logica di **retry-on-empty** e catene di **fallback** sui campi (es. `trailingAnnualDividendYield` vs `dividendYield`).

```

1 def fetch_ticker_info(ticker: str, retries: int = 1) -> dict[str, Any]:
2     cache_key = f"yf_cache:info:{ticker.upper()}"
3     client = get_redis_sync()
4     try:
5         cached = client.get(cache_key)
6         if cached is not None:
7             logger.info("yfinance INFO cache HIT for %s", ticker)
8             return json.loads(cached)

```

```

9      except Exception:
10         pass
11     logger.info("yfinance INFO cache MISS for %s - downloading", ticker)
12     info: dict[str, Any] = {}
13     for attempt in range(retries + 1):
14         try:
15             info = yf.Ticker(ticker).info
16         except Exception:
17             info = {}
18         if info and info.get("regularMarketPrice") is not None:
19             break
20         if attempt < retries:
21             time.sleep(0.8)
22     if info and info.get("regularMarketPrice") is not None:
23         try:
24             client.set(cache_key, json.dumps(info, default=str), ex=
CACHE_TTL_SECONDS)
25         except Exception:
26             pass
27     return info

```

Listing 5: Provider dati di mercato (`market_provider.py`): implementazione del pattern *Cache-Aside* su Redis con logica di *Hit/Miss*, invalidazione tramite TTL e meccanismo di *fallback*.

9.2 Analisi informative per singolo titolo

Oltre alle analisi generative descritte in §11, la dashboard offre un pannello dedicato all'**analisi del singolo titolo** (*Analisi titolo*) che espone tre ulteriori funzionalità di natura puramente **informativa**, basate sui dati restituiti da Yahoo Finance e gestite — coerentemente con il resto del sistema — tramite task Celery asincroni (`fetch_etf_holdings_task`, `fetch_dividends_task`, `fetch_analyst_task`), con risposta HTTP 202, successivo polling e il medesimo strato di caching *cache-aside*:

- **Composizione ETF**: per i soli strumenti di tipo ETF o fondo (`quoteType` pari a `ETF/MUTUALFUND`) restituisce le principali partecipazioni sottostanti (*holdings*); per i titoli azionari ordinari il sistema risponde in modo controllato, segnalando l'assenza di composizione anziché un errore.
- **Dividendi**: espone lo storico delle distribuzioni del titolo; in assenza di dividendi (titolo che non ne distribuisce) viene restituito un esito esplicito.
- **Target analisti**: riporta i prezzi obiettivo aggregati degli analisti (es. valore medio), con gestione esplicita del caso in cui il dato non sia disponibile.

A differenza delle analisi generative, queste funzioni **non** coinvolgono l'LLM: sono pure interrogazioni del provider arricchite da *retry* e catene di *fallback* e, come le analisi AI *per-ticker*, restano **effimere** (nessuna persistenza relazionale), risolte tramite il *Result Backend* di Celery. Insieme alle tre analisi generative (§11), compongono la suite di analisi per singolo titolo accessibile dal pannello *Analisi titolo*.

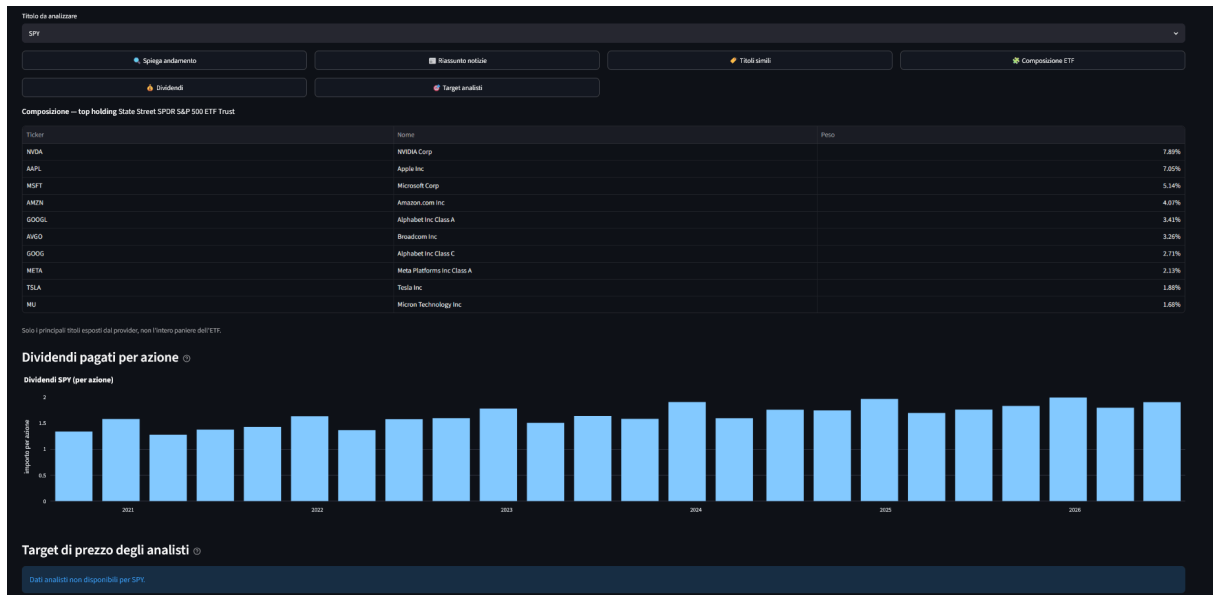


Figura 6: Analisi dividendi, target analisti e composizione ETF (eventuale)

10 Motore quantitativo

Per elevare il sistema da semplice aggregatore CRUD a piattaforma di analisi, è stato introdotto un **Quant Engine** nel livello di background processing, basato sulla libreria **pandas**. A partire dallo storico annuale dei prezzi (utilizzando gli *adjusted close*, rettificati per split e dividendi per non distorcere il calcolo dei rendimenti), il motore calcola in modo deterministico: i **pesi reali** di ciascun asset sul valore di mercato corrente; il **rendimento storico a 1 anno** per asset e per portafoglio; la **volatilità annualizzata** del portafoglio (via matrice di covarianza); il **valore di mercato corrente** e il **profitto/perdita (P/L)** assoluto e percentuale, per posizione e aggregato; metriche **multi-finestra** (rendimenti a 1M, 3M, 6M, 1Y) e indicatori per singolo titolo (massimo drawdown, distanza dai massimi/minimi a 52 settimane), oltre a indicatori fondamentali quali *Price/Earnings* e *Dividend Yield*.

Per la volatilità, il sistema costruisce la matrice di covarianza giornaliera Σ dei rendimenti logaritmici degli asset e, dato il vettore dei pesi $\mathbf{w}$, deriva la varianza di portafoglio $\mathbf{w}^T \Sigma \mathbf{w}$, successivamente annualizzata:

$$\sigma_{\text{annua}} = \sqrt{\mathbf{w}^T \Sigma \mathbf{w}} \sqrt{252}$$

Rispetto a una semplice aggregazione delle deviazioni standard individuali, questo approccio tiene conto delle **correlazioni** fra gli asset, fornendo una stima del rischio di portafoglio rigorosa.

Una nota implementativa rilevante: l'estrazione delle metriche avviene attraverso **due percorsi di calcolo paralleli** — `calculate_portfolio_metrics`, utilizzato dal task di analisi AI, e `build_portfolio_analytics`, utilizzato dalla dashboard. Le metriche multi-finestra sono ricavate per **partizionamento locale** della serie storica annuale già in cache (es. 21 sedute per la finestra mensile), evitando chiamate di rete aggiuntive.

```

1 first = float(series.iloc[0])
2 last = float(series.iloc[-1])
3 return_1y_pct = round((last / first - 1.0) * 100.0, 2) if first else None
4
5 log_ret = np.log(series / series.shift(1)).dropna()
6 annualized_vol_pct = (
7     round(float(log_ret.std() * math.sqrt(252)) * 100.0, 2) if len(log_ret)
8     > 1 else None
9 )

```

```

10 period_high = round(float(series.max()), 4)
11 period_low = round(float(series.min()), 4)
12 running_max = series.cummax()
13 max_drawdown_pct = round(float(((series - running_max) / running_max).min()
14 * 100.0), 2)
15
16 ind = _extract_indicators(info)
17 high_52 = ind["fifty_two_week_high"] if ind["fifty_two_week_high"] is not
18 None else period_high
19 low_52 = ind["fifty_two_week_low"] if ind["fifty_two_week_low"] is not None
20 else period_low
21 pct_from_high = round((last / high_52 - 1.0) * 100.0, 2) if high_52 else
22 None
23 pct_from_low = round((last / low_52 - 1.0) * 100.0, 2) if low_52 else None

```

Listing 6: (quant.py) - Calcolo della volatilità annualizzata via matrice di covarianza.

Grafici & metriche Prezzi & P/L Analisi titoli Report portfolio									
Ticker	Quantità	Prezzo medio	Costo	Settore		Mercato			
AAPL	15	175.50	2,632.50	Technology		US			
TSLA	8	180.20	1,441.60	Consumer Cyclical		US			
KO	40	58.30	2,332.00	Consumer Defensive		US			
JPM	12	165.00	1,980.00	Financial Services		US			
O	35	52.10	1,823.50	Real Estate		US			
SPY	5	460.00	2,300.00	ETF		US			

Ticker	Prezzo attuale	Valore attuale	Costo	P/L	P/L %	Max 1Y	Min 1Y	Volume medio	
AAPL	298.01	4,470.15	2,632.50	1,837.65	69.81%	317.40	198.96	198.96	47,300,354
TSLA	400.49	3,203.92	1,441.60	1,762.32	122.25%	498.83	288.77	288.77	58,082,137
KO	79.39	3,175.60	2,332.00	843.60	36.17%	84.04	65.35	65.35	15,566,679
JPM	325.22	3,902.64	1,980.00	1,922.64	97.10%	338.09	272.11	272.11	9,008,248
O	60.24	2,108.40	1,823.50	284.90	15.62%	67.94	55.86	55.86	5,848,229
SPY	746.74	3,733.70	2,300.00	1,433.70	62.33%	760.40	591.89	591.89	61,627,638

Figura 7: Dashboard con la tabella analitica (P/L, pesi, rendimenti)

11 Intelligenza artificiale come funzionalità

11.1 Il modulo GenerativeAIAnalyzer

La classe **GenerativeAIAnalyzer** (in `services/ai_service.py`) agisce come layer di orchestrazione verso i Large Language Models: recupera l'aggregato relazionale del portafoglio e le metriche calcolate dal Quant Engine, strutturandole in un *System Prompt* pre-calibrato. Il principio progettuale cardine è che **l'LLM non calcoli alcuna metrica**: al modello è demandato esclusivamente il compito di spiegare, sintetizzare e contestualizzare in forma narrativa le metriche prodotte dal sistema, generando un report **puramente informativo** che non costituisce consulenza finanziaria. Data la natura non deterministica e le elevate latenze (TTFB) delle chiamate verso i provider LLM, l'intero processo di inferenza è segregato nel Worker asincrono: il gateway si limita ad accodare la richiesta, delegando al client il polling. Il modulo espone **quattro servizi**: il report di portafoglio e tre analisi *per-ticker* — spiegazione dell'andamento, sintesi delle notizie, individuazione di titoli comparabili (Peers).

11.2 Il pivot tecnologico sul provider e la mitigazione delle allucinazioni

Durante il collaudo del modulo è emersa una criticità nota nei LLM di dimensioni contenute (modelli da circa 8B parametri): l'inclinazione a generare **allucinazioni matematiche**, ovvero a inventare metriche non presenti nel payload (es. Alfa, Beta di Jensen) o ad approssimare scorrettamente i pesi del portafoglio. Per ottimizzare al contempo **latenza e costi** sui task generativi asincroni e garantire la fedeltà numerica dei report, si è proceduto a un **pivot tecnologico** sul provider, affiancato da interventi di prompt engineering:

1. **Pivot del provider e scaling del modello.** Adozione di **GPT OSS 120B**, servito dalla piattaforma di inferenza ad altissime prestazioni **Groq** e interfacciato tramite un SDK compatibile con quello di OpenAI (impiegato esclusivamente per la *forma* del client, **non** come provider). Il precedente modello da 8B presentava problemi di accuratezza sui dati numerici, risolti dal modello più capace.
2. **Prompt Engineering** sul System Prompt: divieto di generare layout tabellari (deman-dati alla UI) e vincolo assoluto di aderenza al dato grezzo fornito dal Quant Engine, con esplicito divieto di citare metriche non presenti nell'input.

La temperatura del modello è stata calibrata a **0.5** per bilanciare determinismo numerico e fluidità narrativa. A tutela dell'utente, a ogni report viene apposto in modo deterministico un **disclaimer** che ne ribadisce la natura puramente informativa.

11.3 Intelligenza effimera e pattern “Genera-poi-Verifica”

Le tre analisi *per-ticker* sono **time-sensitive** e prive di rilevanza per l'audit storico: per questo si è scelto consapevolmente di **non persisterle** nel database relazionale, mantenendole come stato effimero (Result Backend di Celery + `session_state` del client). Per le sole analisi che restano persistenti (il report di portafoglio) vale invece quanto descritto in §6.3.

Per azzerare il rischio di allucinazioni nella proposta di titoli comparabili, è stato ingegneriz-zato il pattern **Genera-poi-Verifica**: l'LLM produce un set euristico di potenziali competitor; il backend intercetta tale output e lo **valida deterministicamente** contro il provider finan-ziario reale, scartando i ticker inesistenti o delistati (privi di `regularMarketPrice`). In questo modo l'interfaccia espone esclusivamente dati riconciliati, confinando l'LLM al ruolo di motore semantico guidato da *System Prompt* altamente restrittivi.

```

1 SYSTEM_PROMPT = (
2     "You are an expert Wall Street Chief Investment Officer (CIO). You receive
3     raw portfolio data (including exact sectors and weights) and advanced
4     quantitative metrics (Returns, Volatility, Max Drawdown, Sharpe Ratio).\n"
5     "Write a comprehensive institutional due-diligence report in Italian, in a
6     professional, assertive advisory tone.\n"
7     "STRICT RULES:\n"
8     "- GOLDEN RULE: Do NOT merely list the assets, percentages, or returns.
9     INTERPRET the data. Correlate the assets.\n"
10    "- FORMATTING RESTRICTIONS: Do NOT use tables, headings (#, ##), or emojis.
11    They break the frontend UI.\n"
12    "- PERMITTED FORMATTING: You MUST use bold text to highlight key
13    financial concepts, risk factors, metric names, and specific ticker symbols
14    .\n"
15    "- Structure the report EXACTLY into these 5 numbered sections (write '1.
16    Panoramica Esecutiva', '2. Analisi Settoriale' etc. as plain text text to
17    separate sections):\n"
18    " 1. Panoramica Esecutiva: Synthesize the core macro-strategy of the
19    portfolio (e.g., core-satellite, value vs growth, barbell).\n"
20    " 2. Analisi Settoriale: Analyze the sector exposure using the provided '
21    sector' strings. Evaluate concentration risks and sector synergies.\n"
22    " 3. Performance dei Singoli Asset: Explain the portfolio return by
23    identifying which assets acted as growth drivers (outperformers) and which
24    as defensive anchors or laggards.\n"
25    " 4. Dinamiche di Rischio: Evaluate the Annualized Volatility, Max
26    Drawdown, and Sharpe Ratio provided in the input. Explain how
27    resilient the portfolio is to stress.\n"
28    " 5. Insight Operativi (Bulleted list with '-'): Provide a concise list of
29    at least 3 tactical rebalancing suggestions or macroeconomic vulnerabilities
30    to monitor.\n"

```

```

14     "- Be concise, dense with information, and consistent. Do NOT add a
        disclaimer yourself: one is appended automatically."

```

Listing 7: (ai_service.py) - System Prompt anti-allucinazione (divieto di tabelle e di metriche non fornite).

```

1 @celery_app.task(name="suggest_peers_task")
2 def suggest_peers_task(ticker: str) -> dict[str, Any]:
3     # [...] Setup iniziale omezzo per brevit 
4
5     # FASE 1: GENERAZIONE (L'LLM propone i competitors)
6     raw = asyncio.run(_run_peer_generation(ticker, name, sector, industry))
7
8     validated: list[dict] = []
9     seen = {ticker}
10
11     # FASE 2: VERIFICA (Mitigazione allucinazioni)
12     for cand in _parse_peer_candidates(raw):
13         if not isinstance(cand, dict): continue
14
15         cand_ticker = str(cand.get("ticker", "")).upper().strip()
16         if not cand_ticker or cand_ticker in seen:
17             continue
18
19         seen.add(cand_ticker)
20         time.sleep(0.3) # Backoff per evitare rate-limiting dal provider
21
22         # Validazione incrociata: il ticker esiste davvero sul mercato?
23         cand_info = fetch_ticker_info(cand_ticker, retries=1) or {}
24         if cand_info.get("regularMarketPrice") is None:
25             continue # Fallimento: l'LLM ha allucinato un ticker invalido ->
                scartato
26
27         validated.append({
28             "ticker": cand_ticker,
29             "name": cand_info.get("longName") or cand_ticker,
30             "sector": cand_info.get("sector"),
31             "current_price": cand_info.get("regularMarketPrice"),
32             "reason": str(cand.get("reason", ""))[:120],
33         })
34
35         if len(validated) >= 6: break # Limite massimo di risultati
36
37     return {"status": "success", "ticker": ticker, "peers": validated}

```

Listing 8: (ai_tasks.py) - Pattern “Genera-poi-Verifica” nel task dei peer.

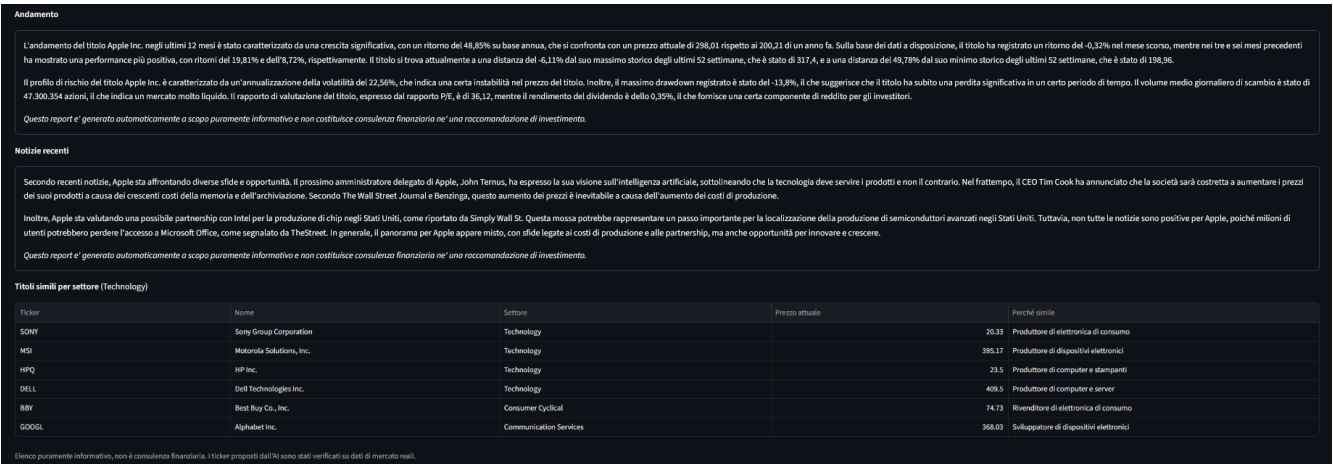


Figura 8: Andamento, riassunto notizie e titoli simili.

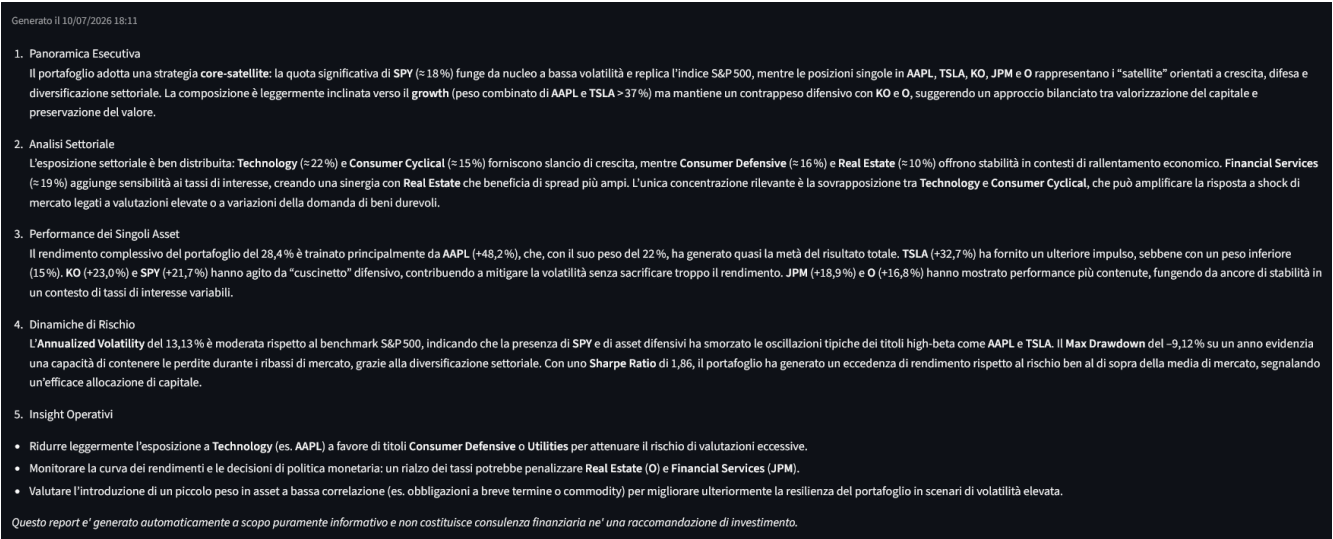


Figura 9: Esempio di report generato dall'AI.

12 Sicurezza e controllo degli accessi

12.1 Autenticazione basata su JWT

Per la gestione dell'identità si è optato per il protocollo **OAuth2 con flusso Password (Bearer Token)**, implementato tramite FastAPI e la libreria PyJWT. Le password sono storizzate tramite hashing unidirezionale con algoritmo **bcrypt**; all'autenticazione, verificate le credenziali, il sistema genera un JWT firmato con l'algoritmo simmetrico **HS256**, con chiave segreta gestita via variabile d'ambiente. Il payload del token contiene il *Subject* (ID utente) e una scadenza (*Expiration Time*). Questo elimina la necessità di interrogare il database per la mera validazione di sessione, garantendo scalabilità orizzontale (*statelessness*). In un deployment multi-servizio, un algoritmo asimmetrico (RS256/ES256) consentirebbe la verifica tramite chiave pubblica senza condividere il segreto di firma.

12.2 Autorizzazione e Data Ownership

L'identità del client (`current_user`) viene iniettata come dipendenza in ogni richiesta protetta. Oltre all'autenticazione, è implementato un meccanismo di autorizzazione basato

sull'**appartenenza delle risorse** (*Data Ownership*): ogni operazione di lettura o mutazione su entità relazionali (portafogli, posizioni, report AI) subisce un controllo incrociato obbligatorio sull'`user_id` proprietario; in caso di mancata corrispondenza vengono restituiti errori standardizzati HTTP 403 (Forbidden) o HTTP 404 (Not Found). Questo paradigma elimina alla radice la vulnerabilità nota come **Insecure Direct Object Reference (IDOR)**.

```

1 async def get_current_user(
2     token: Annotated[str, Depends(oauth2_scheme)],
3     db: Annotated[AsyncSession, Depends(get_session)],
4 ) -> User:
5     credentials_exception = HTTPException(
6         status_code=status.HTTP_401_UNAUTHORIZED,
7         detail="Could not validate credentials",
8         headers={"WWW-Authenticate": "Bearer"},
9     )
10    try:
11        payload = jwt.decode(
12            token, settings.SECRET_KEY, algorithms=[settings.ALGORITHM]
13        )
14        subject = payload.get("sub")
15        if subject is None:
16            raise credentials_exception
17        user_id = uuid.UUID(str(subject))
18    except (InvalidTokenError, ValueError):
19        raise credentials_exception
20    current_user = await crud_user.get(db, user_id)
21    if current_user is None:
22        raise credentials_exception
23    return current_user

```

Listing 9: (deps.py) - `get_current_user` (decodifica JWT → utente)

```

1 async def get_owned_portfolio(
2     portfolio_id: uuid.UUID,
3     current_user: Annotated[User, Depends(get_current_user)],
4     db: Annotated[AsyncSession, Depends(get_session)],
5 ) -> Portfolio:
6     portfolio = await crud_portfolio.get(db, portfolio_id)
7     if portfolio is None:
8         raise HTTPException(
9             status_code=status.HTTP_404_NOT_FOUND, detail="Portfolio not found"
10        )
11    if portfolio.user_id != current_user.id:
12        raise HTTPException(
13            status_code=status.HTTP_403_FORBIDDEN, detail="Not enough
14permissions"
15        )
16    return portfolio

```

Listing 10: (deps.py) - `get_owned_portfolio` (controllo di ownership con 403/404)

12.3 Protezione del gateway (Rate Limiting)

In un'architettura esposta e interfacciata con servizi terzi a pagamento (LLM API) o soggetti a *throttling* (Yahoo Finance), è stato implementato un layer di **Rate Limiting** su Redis (DB 0) tramite `fastapi-limiter`. Le soglie sono calibrate sull'impatto computazionale: gli endpoint di inferenza AI sono limitati a **3 richieste al minuto** per utente (per contenere i costi sulla piattaforma Groq), mentre gli endpoint di recupero dati di mercato operano con una soglia più tollerante di **30 richieste al minuto**, evitando falsi positivi (HTTP 429) durante i cicli di aggiornamento massivo, grazie alla protezione del sottostante strato di caching. Il conteggio è

associato a un *identifier* univoco composto da ID utente (estratto dal JWT) e path della rotta (es. `user:{id}:/ai/report`), per evitare collisioni tra endpoint.

13 Frontend e dashboard

A completamento dell'architettura è stato ingegnerizzato un client frontend totalmente disaccoppiato dal core, containerizzato autonomamente ed esposto sulla `public_net`. La scelta è ricaduta su **Streamlit**, per coniugare rapidità di sviluppo e vocazione analitico-finanziaria. Il frontend opera come **puro client REST**, orchestrando chiamate HTTP (libreria `requests`) verso il servizio API; lo stato (es. il JWT post-login) è gestito dal `session_state` di Streamlit. Coerentemente con i pattern di resilienza del backend, il client intercetta i codici 401 (Unauthorized) forzando il logout; gestisce i codici 429 (Too Many Requests) informando l'utente del rate limiting in corso; e per le operazioni asincrone a lunga durata (fetch dei dati e inferenza AI) implementa un meccanismo di **HTTP Polling** con spinner interattivi, attendendo la transizione del task verso SUCCESS o FAILURE prima di aggiornare i componenti visivi.

Il layer di presentazione utilizza **Plotly** per i grafici (incluso lo storico prezzi a 12 mesi con finestre temporali — 1M/3M/6M/1Y — filtrate lato client via `pandas`) e l'oggetto **pandas Styler** per una formattazione condizionale rigorosa (es. colorazione di guadagni/perdite), disaccoppiando la precisione dei dati dalla loro rappresentazione visiva.

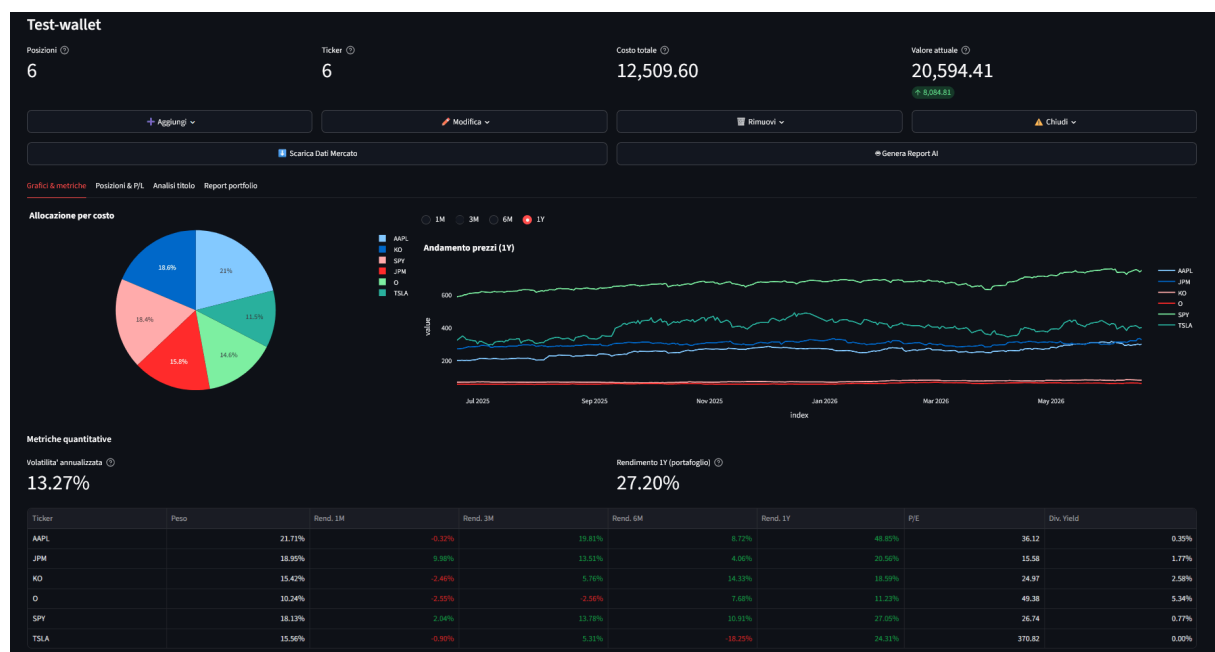


Figura 10: Dashboard - grafici e metriche

14 Observability, qualità e collaudo prestazionale

14.1 Structured Logging (JSON)

L'intero sistema di logging emette **log strutturati in formato JSON** (tramite `python-json-logger`). L'implementazione richiede un *hijacking* esplicito dei logger interni del server ASGI (`uvicorn`) e l'aggancio ai segnali di inizializzazione (`setup_logging`) del Worker Celery, così che ogni componente produca log omogenei con chiavi standardizzate (`timestamp`, `level`, `message`) e metadati di dominio (es. `portfolio_id`), abilitando ricerca avanzata e alerting su stack centralizzati (ELK, Datadog).

14.2 Esportazione delle metriche (Prometheus e Grafana)

È stato integrato un layer di strumentazione per **Prometheus** (`prometheus-fastapi-instrumentator`) che espone l'endpoint `/metrics` con le *RED Metrics* (Rate, Errors, Duration) del traffico HTTP e le statistiche interne del processo. L'endpoint è escluso dalla documentazione OpenAPI applicativa; si precisa che la mera esclusione **non costituisce una misura di sicurezza** (in produzione andrebbe raggiungibile solo dalla rete di monitoring). Un'istanza **Prometheus** containerizzata esegue lo *scraping* periodico (ogni 5 secondi) e **Grafana** funge da cruscotto, adottando la dashboard standard *FastAPI Observability* (ID 22676), che mappa throughput (RPS), latenza per endpoint (percentili) e tassi di errore.

14.3 Testing automatizzato e Continuous Integration

È stata implementata una suite di **test di integrazione** con `pytest` e `pytest-asyncio`. Le scelte di isolamento sono: un database PostgreSQL **effimero dedicato** (`finsight_test`) anziché in-memory, poiché l'architettura vincola nativamente funzionalità specifiche di PostgreSQL (JSONB, UUID); e la simulazione delle richieste HTTP *in-memory* tramite `httpx` con `ASGITransport`, aggirando il binding di rete reale. Questo permette di testare i flussi di autenticazione e i controlli di ownership in tempi dell'ordine dei millisecondi.

Una pipeline di **Continuous Integration** (GitHub Actions), attivata a ogni `push` o `pull request` verso `main`, esegue in sequenza: l'istanziamento di un ambiente con i servizi core (PostgreSQL e Redis); le migrazioni `alembic upgrade head` (validando il determinismo dello schema); l'intera suite `pytest`. Qualsiasi anomalia interrompe la pipeline, realizzando un rigoroso *gatekeeping* qualitativo.

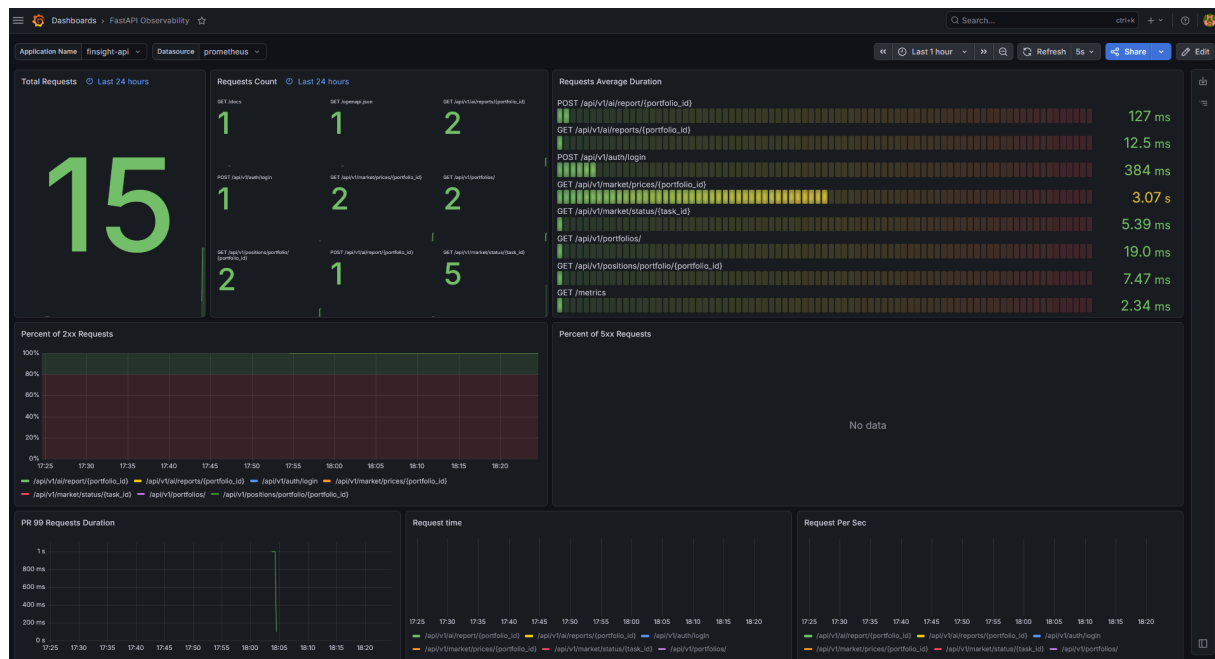


Figura 11: Dashboard Grafana con i vari parametri, in condizioni normali.

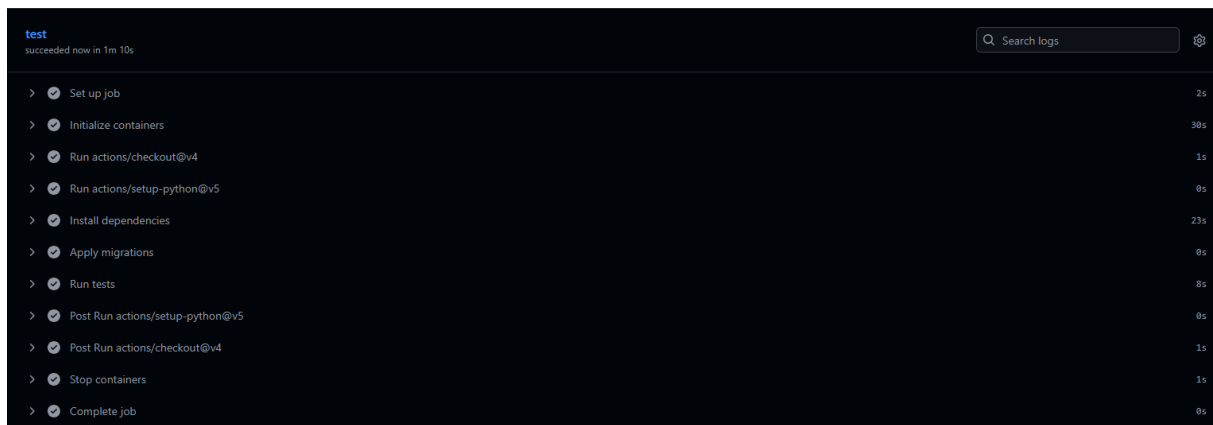


Figura 12: Pipeline CI su GitHub Actions.

14.4 Chaos Engineering e diagnosi del collo di bottiglia

Conclusa l’implementazione delle funzionalità e dichiarato il *code freeze* sulla logica applicativa, l’attenzione si è spostata sulle *Day-2 Operations*. È stato sviluppato uno script di **Load Testing** con il framework **Locust**, progettato per simulare una congestione asimmetrica: **500 utenti concorrenti** con *spawn rate* di 50 utenti/secondo, con un **task pesante** (peso 3) di chiamate massicce a `POST /api/v1/auth/login` con credenziali errate — per forzare ripetutamente l’algoritmo `bcrypt`, intensivo a livello di CPU — e un **task leggero** (peso 1) verso `GET /health`. Per garantire l’efficacia del test, lo script (tramite l’hook `test_start`) registra preliminarmente l’utente target, evitando il rigetto istantaneo per “utente inesistente”.

Le misurazioni raccolte durante la saturazione hanno rivelato un comportamento critico tipico delle applicazioni asincrone. Nonostante la CPU del container si mantenesse intorno al **10–11%** (saturazione di un singolo core logico), il sistema ha mostrato un collasso generalizzato della reattività: la latenza media dell’endpoint `/health`, pur eseguendo un’operazione da pochi millisecondi, è schizzata a **27,3 secondi**, con picchi al 99° percentile oltre i 100 secondi.

Diagnosi (Starvation dell’Event Loop): il degrado non è stato causato dall’esaurimento globale delle risorse di calcolo, bensì dalla paralisi del thread principale. La funzione `bcrypt`, essendo strettamente sincrona e CPU-bound, ha **sequestrato l’unico event loop** di `FastAPI/Uvicorn`; di conseguenza tutte le richieste concorrenti — incluse quelle banali dirette a `/health` — sono state accodate in attesa che il thread si liberasse. L’analisi ha confermato la solidità della rete e l’efficacia del rate limiting (i fallimenti si sono concentrati sui **401 Unauthorized** voluti dal test), evidenziando la necessità di un intervento di **Thread Offloading** sulle operazioni crittografiche.

14.5 Ottimizzazione asincrona (Thread Offloading) e validazione comparativa

La *remediation* ha interessato il layer crittografico (`backend/app/core/security.py`), nel rispetto del vincolo di non alterare la logica applicativa. Le funzioni di hashing (`verify_password`, `get_password_hash`), originariamente sincrone, sono state rifattorizzate in **coroutine asincrone**, esternalizzando il carico CPU-bound di `passlib (bcrypt)` rispetto all’event loop tramite il costrutto `run_in_threadpool`. Le computazioni vengono così delegate a un pool di thread, liberando istantaneamente il thread principale per l’accettazione di nuove richieste; l’adeguamento è stato propagato ai controller di autenticazione e al repository utente.

Il collaudo di validazione, replicando le esatte condizioni di carico (500 utenti, focus sul login), ha confermato il successo dell’intervento:

- **Failure rate azzerato:** gli errori di timeout/sovraccarico (HTTP 5xx) sono scesi dal **33%** allo **0%**; il sistema ha gestito **11.888 richieste** (contro le precedenti 3.554 a parità di tempo), tutte con successo.
- **Throughput e parallelismo:** l'utilizzo CPU del processo è passato dal circa 10% (singolo core) al **circa 150%**, prova che l'hashing viene ora eseguito in parallelo su più core; le richieste al secondo (RPS) sono aumentate di un fattore **5×**.
- **Abbattimento della latenza:** la latenza mediana aggregata è crollata da **33 s a 3,9 s**, con riduzioni fino a **15×** sui percentili più alti (99° percentile sceso da 112 s a 7,3 s per gli endpoint leggeri).

Shift del collo di bottiglia. Un'evidenza di particolare interesse ingegneristico: eliminata la starvation, l'endpoint `/health` non è tornato a latenze dell'ordine dei millisecondi ma si è stabilizzato su una mediana di circa 3,7 s, quasi identica a quella di `/login`. Il collo di bottiglia si è dunque **spostato**: il sistema ha raggiunto il limite del *throughput grezzo* del singolo processo `Uvicorn` e del relativo pool di connessioni. Essendo il carico offerto (circa 250 RPS) nettamente superiore alla capacità di smaltimento del singolo processo (circa 63 RPS), si è formata una coda di attesa uniforme condivisa equamente da tutti gli endpoint; sotto la soglia di saturazione il sistema risponde in millisecondi. Il risultato certifica la robustezza dell'implementazione: portato oltre i limiti fisici, il sistema degrada in modo **aggraziato** (graceful degradation), senza perdere una singola richiesta.

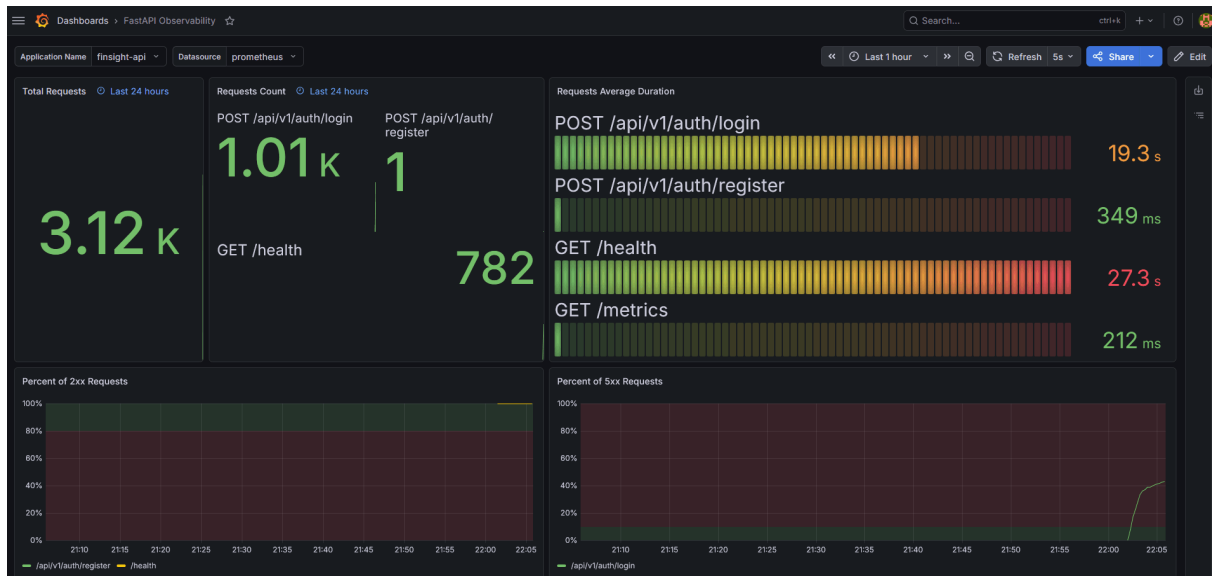


Figura 13: Confronto Grafana (*prima*) - thread offloading (latenza di `/health` e RPS)

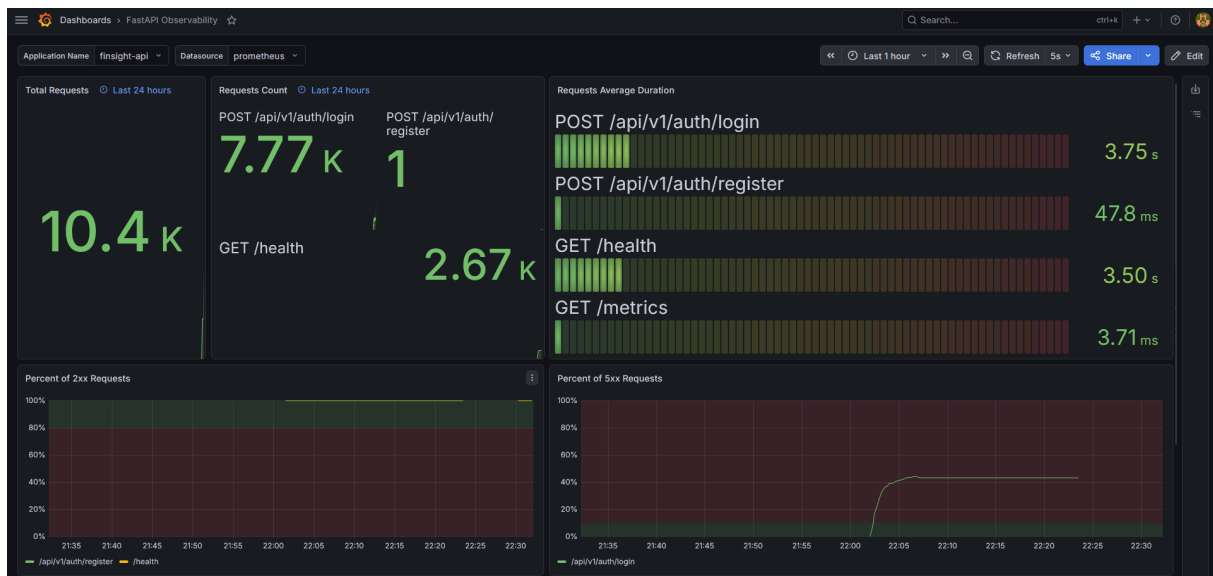


Figura 14: Confronto Grafana (*dopo*) - thread offloading (latenza di `/health` e RPS)

15 Metodologia di sviluppo assistita da AI

In conformità con le direttive del progetto, lo sviluppo di *FinSight* ha previsto l'utilizzo di strumenti di AI generativa come *coding assistant*, sotto stretta supervisione architetturale. È importante distinguere questa accezione — l'AI **usata per costruire** il progetto — dall'AI **come funzionalità del prodotto** descritta in §11.

La metodologia adottata, definibile come **Orchestrazione Multi-Agente (Dual-AI Pattern)**, ha segregato il ciclo di vita del software tra tre attori fondamentali. Il **Coordinatore Umano** (detentore della visione di dominio, responsabile delle code-review e del collaudo) e l'**AI Tech Lead & Architetto** (Google Gemini 3.1 Pro) hanno operato in stretta sinergia attraverso un continuo e rigoroso dibattito critico. Questa interazione non si è configurata come una mera delega passiva, ma come un confronto dialettico volto a sfidare le premesse iniziali, analizzare i rischi di implementazione, esplorare pattern alternativi e neutralizzare il bias cognitivo dell'*Echo Chamber*.

Solo al termine di questo processo congiunto di studio e validazione delle opzioni, le decisioni ingegneristiche venivano cristallizzate in capitolati tecnici formali. Tali direttive (Prompt Master) venivano infine somministrate al terzo attore, l'**AI Esecutore** (Anthropic Claude 4.8 Opus), deputato in totale isolamento alla sola generazione del codice. Questa rigorosa separazione cognitiva ha contribuito a mitigare i noti anti-pattern degli LLM, in particolare il degrado del contesto su codebase estese. La raccolta curata dei capitolati somministrati all'esecutore è riportata integralmente nell'**Appendice A** (prompt-ai), come dichiarazione di trasparenza metodologica.

16 Limiti attuali

- **Single-process / single-instance:** il collaudo di §14.5 ha evidenziato che la capacità del singolo processo *Uvicorn* (circa 63 RPS) costituisce il limite di throughput; la scalabilità richiederebbe più repliche dietro un load balancer.
- **Centralizzazione di Redis:** una sola istanza Redis assolve quattro responsabilità (broker Celery, result backend, cache e rate limiting), già segregate su database logici distinti

(DB 0 per cache/rate limiting; DB 1 e DB 2 per broker/result backend). In produzione converrebbe isolarle su istanze dedicate.

- **MarketDataSnapshot non ancora sfruttato per le analytics storiche:** lo store JSONB funge da audit trail dei payload provider, ma la pipeline analitica legge dalla cache; analytics storiche intensive richiederebbero una tabella normalizzata (es. `HistoricalPrice`).
- **/health minimale:** verifica solo PostgreSQL; probe verso Redis e Worker completerebbero il readiness check.
- **Firma JWT simmetrica (HS256):** adeguata a singolo servizio; uno scenario multi-servizio beneficerebbe di un algoritmo asimmetrico (RS256/ES256).
- **Dipendenza da un provider gratuito:** `yfinance` è soggetto a instabilità di schema e *throttling*; mitigato da cache, fallback e retry, ma non eliminato.

17 Sviluppi futuri

- **Scalabilità orizzontale** del gateway (più repliche `api` dietro reverse proxy/load balancer) e del Worker.
- **Isolamento dei datastore Redis** per responsabilità (code vs cache).
- **Tabella normalizzata delle osservazioni storiche** (`HistoricalPrice`) per abilitare analisi storiche intensive direttamente sul database.
- **Readiness check completo** (probe Redis e Worker) e *liveness/readiness* distinti.
- **Migrazione a firma JWT asimmetrica** in ottica multi-servizio.
- **Arricchimento delle fonti di mercato** (provider alternativi o a pagamento) con strategia multi-provider.
- **Hardening della superficie di monitoring** (protezione di rete/policy per `/metrics`, dashboard di alerting).

18 Conclusioni

FinSight realizza una piattaforma full-stack per la gestione e l'analisi di portafogli che coniuga correttezza di dominio (precisione finanziaria, proprietà ACID), reattività (disaccoppiamento asincrono e pattern 202/polling), sicurezza (JWT, ownership, rate limiting) e qualità operativa (log strutturati, metriche, test, CI). L'inquadramento come **Monolite Modulare Containerizzato** — codebase unica, esecuzione distribuita tra gateway sincrono e motore asincrono — ha permesso di ottenere mantenibilità e scalabilità del worker senza la complessità di un'architettura a servizi indipendenti. Il percorso di *Day-2 Operations* (osservabilità, stress test, diagnosi della starvation dell'event loop e successivo thread offloading) ha inoltre fornito una validazione empirica della robustezza del sistema, che sotto carico estremo degrada in modo aggraziato senza perdita di richieste.

Appendice A — Dichiarazione di utilizzo dell'AI generativa

Si rimanda al documento `prompt-ai`, che raccoglie la metodologia di sviluppo assistita da AI (Orchestrazione Multi-Agente / Dual-AI Pattern) e i capitoli tecnici (Prompt Master) somministrati all'AI esecutrice per ciascuna fase di sviluppo.